

Practical : 11	Implementation of Floyd-Warshall Algorithm for All-Pairs Shortest Path.
Date : 12/08/2026	

Objective :- To implement the Floyd-Warshall Algorithm in C++ to compute the shortest path between every pair of vertices in a weighted directed graph, and to analyse its time and space complexity.

Program :-

```
#include <iostream>
#include <vector>
#include <climits>
#include <iomanip>
using namespace std;

const int INF = 1e9;

// ----- Floyd-Warshall Algorithm (All-Pairs Shortest Path,  $O(V^3)$ ) -----
void floydWarshall(vector<vector<int>>& dist, int V) {
    for (int k = 0; k < V; k++)
        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                if (dist[i][k] < INF && dist[k][j] < INF &&
                    dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}

int main() {
    int V = 5;
    vector<vector<int>> dist = {
        {0, 3, INF, 7, INF},
        {8, 0, 2, INF, INF},
        {5, INF, 0, 1, INF},
        {2, INF, INF, 0, INF},
        {INF, INF, INF, 4, 0}
    };

    floydWarshall(dist, V);

    cout << "All-Pairs Shortest Path Distance Matrix:\n";
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] >= INF) cout << setw(6) << "INF";
            else cout << setw(6) << dist[i][j];
        }
        cout << "\n";
    }
    return 0;
}
```

}

Output :-

All-Pairs Shortest Path Distance Matrix:

0	3	5	6	INF
5	0	2	3	INF
3	6	0	1	INF
2	5	7	0	INF
6	9	11	4	0

Analysis:-

Time & Space Complexity (V = vertices, E = edges):

Method	Time Complexity	Space Complexity
Floyd-Warshall - Dynamic Programming	$O(V^3)$	$O(V^2)$
Dijkstra from every vertex ($V \times O(E \log V)$)	$O(VE \log V)$	$O(V + E)$

Floyd-Warshall computes shortest paths between every pair of vertices simultaneously using dynamic programming: for each intermediate vertex k , it checks whether routing through k shortens the path between every pair (i, j) , giving three nested loops over V vertices and hence $O(V^3)$ time. Because the algorithm works directly on a $V \times V$ distance matrix that is updated in place, its space cost is $O(V^2)$, and it naturally handles negative edge weights (though not negative cycles) since it never relies on a greedy shortest-first choice the way Dijkstra does. For the 5-vertex sample graph above, which contains an unreachable vertex (no path exists into vertex 4 in this configuration, shown as INF), the algorithm correctly leaves those matrix entries as INF while still computing shortest distances of 1 to 11 for every reachable pair, including multi-hop shortcuts such as $2 \rightarrow 4$ costing 6 rather than the direct non-edge. Running Dijkstra's algorithm from each of the V vertices individually would cost $O(VE \log V)$ overall, which is typically faster than $O(V^3)$ on sparse graphs ($E \ll V^2$) but Floyd-Warshall remains simpler to implement and preferable on dense graphs or when all-pairs distances are needed at once.

Conclusion :- The Floyd-Warshall Algorithm was successfully implemented in C++ using a dynamic-programming relaxation over all intermediate vertices to compute all-pairs shortest paths on a weighted directed graph. The program correctly produced the complete distance matrix for the sample graph, including correctly marking the unreachable pair as INF, confirming the expected $O(V^3)$ time and $O(V^2)$ space behaviour. Floyd-Warshall is the preferred choice when shortest distances between every pair of vertices are needed at once or when the graph is dense, while running Dijkstra's algorithm from each source is more efficient for sparse graphs needing all-pairs distances.